Exception Handling in Java

try, catch, finally, throw, throws, custom exceptions,
try-with-resources - explained with live demos

Agenda

- What is an exception? Error vs exception
- Exception class hierarchy (API class diagram)
- Checked vs unchecked exceptions; common exceptions
- How the JVM reacts; exception propagation
- try / catch, multiple catch, catch order, multi-catch
- throw, throws, custom exceptions
- finally, try-with-resources
- Best practices, mistakes, practice questions

What is an Exception?

- An exception is an unexpected event during execution that disturbs the normal flow
- Analogy: you drive to office and the road is blocked - you need a plan B
- Without handling: the program stops abruptly and later lines never run
- With handling: the program recovers, shows a friendly message and continues
- Kinds of problems:
  - Compile-time errors: syntax mistakes, found by javac (missing ;)
  - Runtime errors: exceptions, found while running (divide by zero, null)
  - Logical errors: wrong result, no crash (using + instead of -)
- Exception handling = a structured way to detect, report and recover from runtime problems

Demo: Ex1 - NullPointerException

```
package cg;

public class Ex1 {
    public static void main(String[] args) {
        String str = null;                        // no object
        System.out.println(str.equals("hello")); // calling a method on null
    }
}

// Fix: compare from the literal, which is never null
//   System.out.println("hello".equals(str));    // false
```

Key points

- null means the reference points to no object
- Calling any method on null throws NullPointerException
- It is unchecked: the compiler does not force you to handle it
- Defensive habit: "literal".equals(variable)

Output:

```
Exception in thread "main" java.lang.NullPointerException
        at cg.Ex1.main(Ex1.java:7)
```

Demo: Exp1 - Divide by Zero (unhandled)

```
package cg;

public class Exp1 {
    public static void main(String args[]) {
        int a = 100;
        int b = a / 0;                  // ArithmeticException here
        System.out.println(b);          // never executed
        System.out.println("The End");  // never executed
    }
}
```

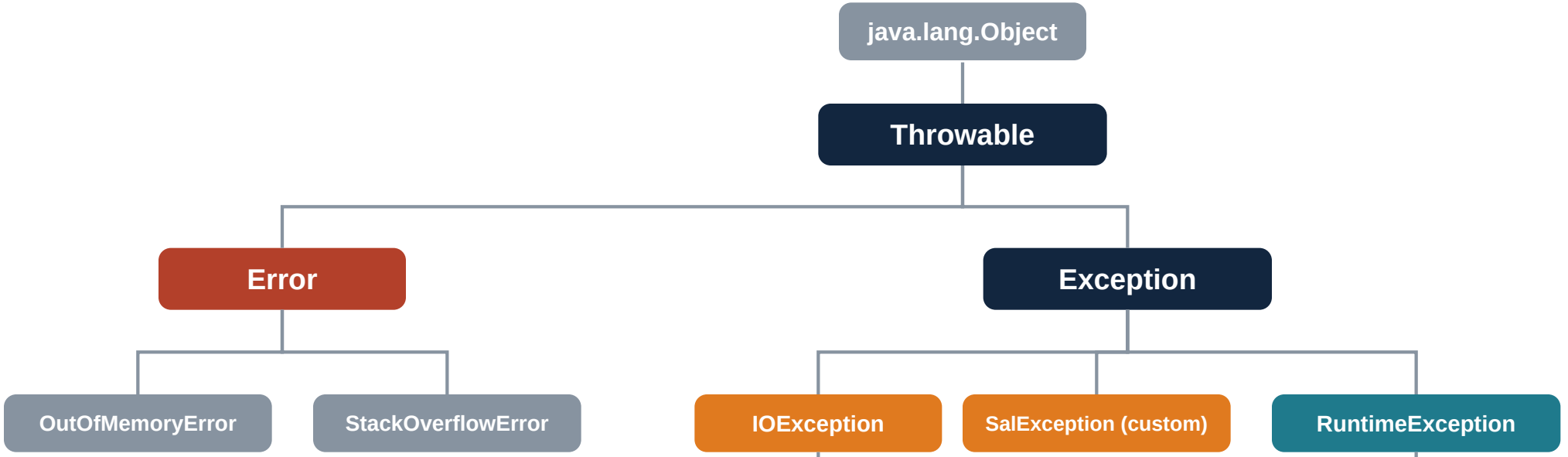
Key points

- Integer division by zero throws ArithmeticException
- Nothing handles it, so the JVM's default handler prints the stack trace and STOPS the program
- "The End" is never printed - this is the problem exception handling solves

Output:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at cg.Exp1.main(Exp1.java:7)
```

Exception API - Class Hierarchy



- Error - serious JVM problems, do not catch
- Checked - compiler forces handling
- Unchecked (RuntimeException) - optional

NullPointerException
ArithmeticException
ArrayIndexOutOfBoundsException
NumberFormatException
InputMismatchException

Checked vs Unchecked Exceptions

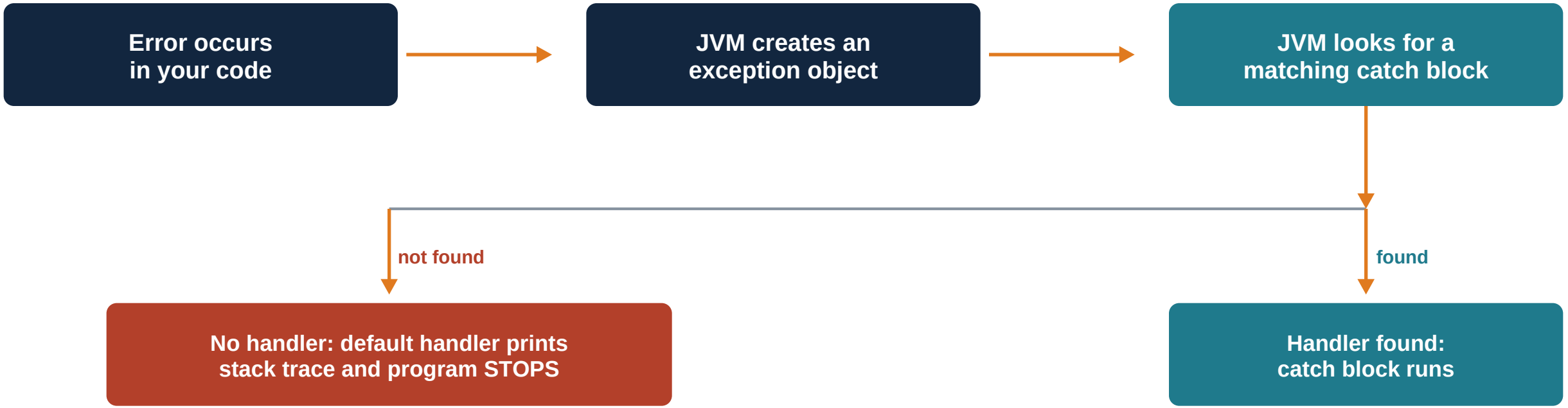
Feature	Checked	Unchecked
Parent class	Exception (not RuntimeException)	RuntimeException
Verified by	compiler (compile time)	not verified by compiler
Must handle?	YES: try-catch or throws	No (but you may)
Typical cause	outside world: files, network	programming bugs
Examples	IOException, FileNotFoundException	NullPointerException, ArithmeticException
Our demos	Exp5_1, Exp5_2, SalException	Ex1, Exp1, Exp5

- Error (OutOfMemoryError, StackOverflowError) is also unchecked, but you normally do not handle it

Common Exceptions You Will Meet

Exception	Typical cause	Demo
NullPointerException	method call on a null reference	Ex1, Exp5
ArithmeticException	integer divide by zero	Exp1, Exp2
NumberFormatException	parseInt("aa")	Exp1_1, Exp3
ArrayIndexOutOfBoundsException	args[0] when no argument given	Exp1_1, Exp3
FileNotFoundException	file name does not exist	FinallyExampleMain
IOException	read/write failure on a stream	FinallyExampleMain
InputMismatchException	Scanner.nextInt() on text	MultiCatchMain

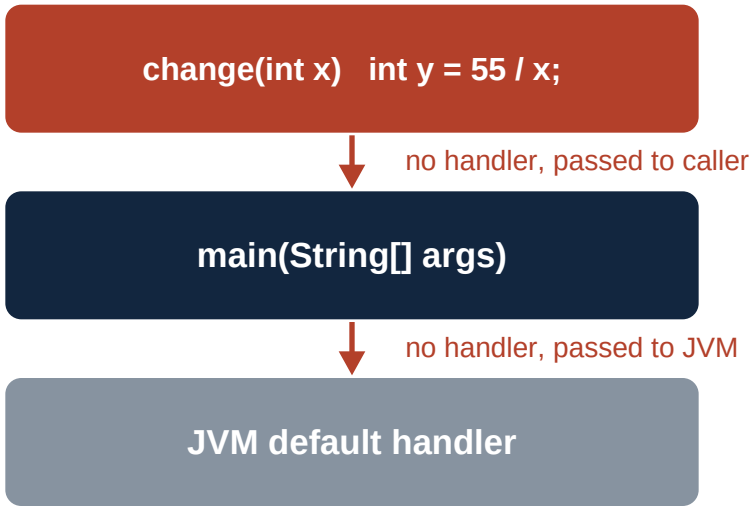
What the JVM Does When an Exception Occurs



- The exception object holds the type, the message and the stack trace
- With a handler the program continues after the try-catch; without one it terminates

Exception Propagation - Call Stack

Call stack (top = running method)




```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at cg.Exp1_1.change(Exp1_1.java:7)
    at cg.Exp1_1.main(Exp1_1.java:18)
```

- If a method does not handle an exception, it travels UP the call chain
- The stack trace lists the path: where it happened (top) and who called it
- Reading it: top line = cause, lines below = callers, bottom = main

Demo: Exp1_1 - Propagation with Arguments

```
package cg;

public class Exp1_1 {
    public static void change(int x) {
        int y = 55 / x;                     // ArithmeticException if x == 0
    }
    public static void main(String args[]) {
        int a = 0;
        a = Integer.parseInt(args[0]);      // may throw 2 exceptions
        change(a);
        System.out.println("The End");
    }
}
// Run:  java cg.Exp1_1 5   /  aa   /  (nothing)  /  0
```

Key points

- One program, four outcomes depending on the input
- None of them is handled, so each one crashes the program
- The next slide lists each case

Exp1_1 - Four Inputs, Four Results

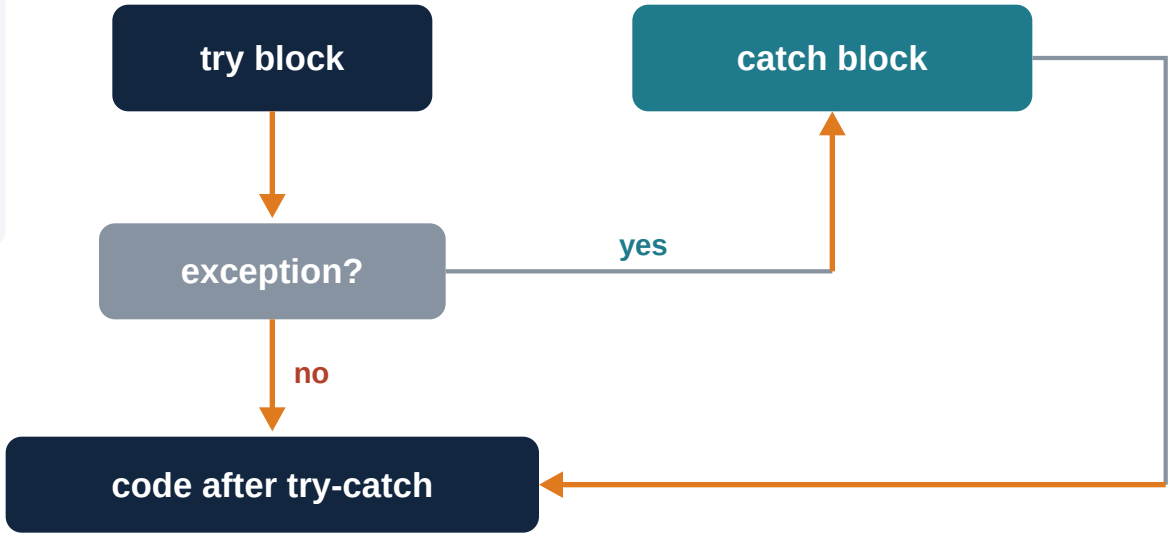
args[0]	Exception thrown	Message
5	none	prints The End
aa	NumberFormatException	For input string: "aa"
(nothing)	ArrayIndexOutOfBoundsException	Index 0 out of bounds for length 0
0	ArithmeticException	/ by zero (from change() to main())

- parseInt fails before change() is even called when the input is not a number

try - catch: Syntax and Flow

```
try {
    // risky code - monitored for exceptions
} catch (ExceptionType e) {
    // handler: runs only if that exception occurs
}
// execution continues here
```

- try monitors the code inside it
- If no exception: catch is skipped
- If exception: rest of try is skipped, matching catch runs



Demo: Exp2 - Handling with try-catch

```
public class Exp2 {
    public static void main(String args[]) {
        int a = 100;
        int b = 0;
        try {                                   // monitor the error code
            b = a / 0;
            System.out.println(b);              // skipped
        } catch (ArithmeticException e) {       // handling exception
            System.out.println("Can not devide by zero");
            e.printStackTrace();                // prints trace, program goes on
        }
        System.out.println("The End");          // now it IS executed
    }
}
```

Key points

- The program no longer crashes
- Lines after the exception inside try are skipped
- e.printStackTrace() prints type, message and path
- Compare with Exp1: now "The End" appears

Output:

```
Can not devide by zero
java.lang.ArithmeticException: / by zero  (stack trace)
The End
```

Demo: Exp3 - Multiple catch Blocks

```
public static void change(int x) { int y = 55 / x; }

public static void main(String args[]) {
    int a = 0;
    try {
        a = Integer.parseInt(args[0]);
        change(a);
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println(e.getMessage());  e.printStackTrace();
    } catch (NumberFormatException e) {
        System.out.println(e.getMessage());  e.printStackTrace();
    } catch (ArithmeticException e) {
        System.out.println(e.getMessage());  e.printStackTrace();
    }
    System.out.println("The End");
}
```

Key points

- One try can have many catch blocks
- Only the FIRST matching catch runs
- Different problems get different responses
- e.getMessage() returns the short description
- Works for propagated exceptions too: change() throws, main catches

Output:

```
no argument -> Index 0 out of bounds for length 0
"aa"        -> For input string: "aa"
"0"         -> / by zero            then: The End
```

Demo: Exp4 - Order of catch Blocks

```
try {
    a = Integer.parseInt(args[0]);
    change(a);
}
catch (ArrayIndexOutOfBoundsException e) {   // subclass FIRST
    System.out.println("ArrayIndexOutOfBoundsException catch block");
    System.out.println(e.getMessage());
}
catch (Exception e) {                        // superclass LAST
    System.out.println("Common catch block");
    e.printStackTrace();
}
System.out.println("The End");
```

Key points

- Specific (child) exceptions first, general (parent) last
- Exception catches everything below it in the hierarchy
- Reverse order = compile error: exception already caught
- Last catch(Exception e) acts as a safety net

Output:
```
no argument -> ArrayIndexOutOfBoundsException catch block
"aa" or "0" -> Common catch block (stack trace)
```

Demo: Exp5 - throw (unchecked)

```
public static void change(int salary) {
    if (salary <= 2000) {                       // custom business rule
        // throw raises an exception manually
        throw new NullPointerException("Enter proper salary");   // unchecked
    }
}
public static void main(String args[]) {
    int a = 0;
    try {
        a = Integer.parseInt(args[0]);
        change(a);
    } catch (Exception e) {
        System.out.println("Caught in main method");
        System.out.println(e.getMessage());
        e.printStackTrace();
    }
    System.out.println("The End");
}
```

Key points

- throw + an exception object = raise it yourself
- Used for business rules, not just JVM errors
- Unchecked: no throws clause needed, compiler does not check
- (A better choice here would be IllegalArgumentException)

Output:

```
args = 1000 -> Caught in main method
                Enter proper salary
                The End
```

Demo: Exp5_1 - Checked: Handle Inside

```
public static void change(int salary) {
    try {
        if (salary <= 2000) {
            throw new Exception("Invalid salary");   // CHECKED exception
        }
    }
    catch (Exception e) {
        System.out.println("Caught inside change method");
    }
}
// main calls change(a) inside its own try-catch,
// but never sees the exception: it was already handled.
```

Output:

```
args = 1000 -> Caught inside change method
                The End
```

Key points

- Exception (not RuntimeException) is checked
- The compiler forces a choice: handle it here OR declare throws
- Option 1: try-catch inside the method (this slide)
- Because it is handled inside, main's catch is never reached

Demo: Exp5_2 - Checked: throws Clause

```
// throws warns callers: "this method may raise a checked exception"
public static void change(int salary) throws Exception {
    if (salary <= 2000) {
        throw new Exception("Invalid salary");        // checked
    }
}
public static void main(String args[]) {
    int a = 0;
    try {
        a = Integer.parseInt(args[0]);
        change(a);                                    // caller must handle
    } catch (Exception e) {
        System.out.println("Caught in main method");
        System.out.println(e.getMessage());
    }
    System.out.println("The End");
}
```

Key points

- Option 2: declare throws in the method signature
- The responsibility moves to the caller
- Without try-catch or throws: compile error "unhandled exception"
- Remove the try-catch in main -> main must declare throws too

Output:

```
args = 1000 -> Caught in main method
               Invalid salary
               The End
```

throw vs throws

Feature	throw	throws
Meaning	raises an exception object now	declares possible exceptions
Where	inside a method body	method signature
Followed by	an object: new Exception("x")	class name(s): Exception
How many	one at a time	one or many (comma separated)
Example	throw new SalException(...)	void change() throws SalException

- Memory trick: throw = action, throws = warning label

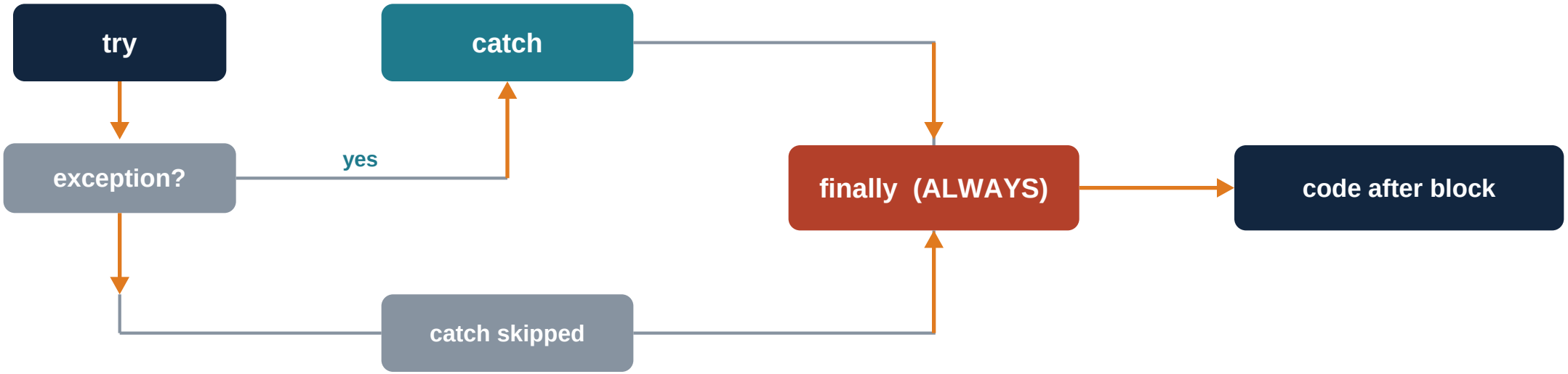
Custom Exception - SalException

```
package cg;

// user defined exception: extends Exception => CHECKED
public class SalException extends Exception {
    private double sal;

    public SalException(String msg, double sal) {
        super(msg);              // passes the message to Exception
        this.sal = sal;
    }
    @Override
    public String toString() {
        return "SalException [sal=" + sal + "]" + super.getMessage();
    }
}
// usage:  throw new SalException("Invalid salary", salary);
```

Key points

- Create your own when built-in ones do not describe your problem
- extends Exception = checked; extends RuntimeException = unchecked
- Extra fields (sal) carry useful data
- Name it clearly: it appears in logs and stack traces

finally - Theory and Flow



- finally runs whether an exception occurs or not (even after return, or if uncaught)
- Use it for cleanup: closing files, connections, releasing resources
- Needs a try block; catch is optional: try-finally is valid

Demo: Exp6 - finally with Custom Exception

```
public static void change(int salary) throws SalException {
    try {
        if (salary <= 2000) {
            throw new SalException("Invalid salary", salary);
        }
    } finally {
        System.out.println("Change method finally");
    }
}
public static void main(String args[]) {
    try {
        a = Integer.parseInt(args[0]);
        change(a);
    } catch (Exception e) {
        System.out.println("Caught in main method");
        System.out.println(e.getMessage());
    } finally {
        System.out.println("main method finally");
    }
    System.out.println("The End");
}
```

Key points

- Inner finally runs first, then the exception reaches main
- Outer finally runs even after the catch
- With a valid salary both finally blocks still run

Output:

```
args = 1000: Change method finally
  Caught in main method
  Invalid salary
  main method finally
  The End
```

Demo: FinallyExampleMain - Closing a File

```
InputStream in = null;
try {
    System.out.println("About to open a file");
    in = new FileInputStream("missingfile.txt");  // FileNotFoundException
    System.out.println("File open");
    int data = in.read();                         // IOException
    while (data != -1) { System.out.print((char) data); data = in.read(); }
} catch (FileNotFoundException e) {               // child before parent
    System.out.println(e.getMessage());
} catch (IOException e) {
    System.out.println("Error reading file");
} finally {
    try {
        System.out.println("file is closed finally");
        if (in != null) { in.close(); }           // close() can throw too
    } catch (IOException e) { System.out.println("Failed to close file"); }
}
```

Key points

- A file is a resource: always close it
- finally guarantees the close, even on error
- The null check avoids a NullPointerException when opening failed
- FileNotFoundException is a child of IOException, so it comes first
- This is a lot of code - Java 7 simplified it (next slide)

Output:

```
file missing: About to open a file
   missingfile.txt (No such file or directory)
   file is closed finally
file exists: contents are printed, then closed
```

Demo: TryWithResources (Java 7+)

```
System.out.println("About to open a file");
try (InputStream in = new FileInputStream("missingfile.txt");) {
    System.out.println("File open");
    int data = in.read();
    while (data != -1) {
        System.out.print((char) data);
        data = in.read();
    }
    // in.close() is called AUTOMATICALLY when the block ends
} catch (FileNotFoundException e) {
    System.out.println(e.getClass().getName());
    System.out.println("Quiting");
} catch (IOException e) {
    System.out.println(e.getClass().getName());
    System.out.println("Quiting");
}
```

Key points

- Resource declared in try (...) is closed automatically
- Works for classes that implement AutoCloseable (close() method)
- No finally and no null check needed
- Closing happens even if an exception is thrown
- Much shorter than the previous demo

Output:

```
file missing: About to open a file
   java.io.FileNotFoundException
   Quiting
```

Demo: MultiCatchMain - Multi-catch + Two Resources

```
try (InputStream in = new FileInputStream("missingfile.txt");
     Scanner s = new Scanner(in);) {          // two resources
    System.out.println("File open");
    int data = s.nextInt();                   // text found, not a number
    System.out.println(data);
} catch (InputMismatchException | IOException e) {   // multi-catch
    System.out.println(e.getClass().getName());
    System.out.println("Quiting");
}
```

Key points

- Multi-catch: one handler for several exception types, separated by |
- The types must NOT be parent and child of each other
- Several resources are separated by ; and closed in REVERSE order (Scanner, then stream)
- The file starts with the word "This", so nextInt() fails

Output:

```
file exists: File open
   java.util.InputMismatchException
   Quiting
```

Exception Keywords at a Glance

Keyword	Purpose	Demo
try	wraps risky code being monitored	Exp2 ... Exp6
catch	handles a specific exception type	Exp3, Exp4
finally	cleanup that always runs	Exp6, FinallyExampleMain
throw	raises an exception manually	Exp5, Exp5_1, Exp5_2
throws	declares checked exceptions of a method	Exp5_2, Exp6
try ( )	auto-closes resources (Java 7)	TryWithResources, MultiCatchMain

Best Practices

- Catch specific exceptions first; use Exception only as the last safety net
- Never leave a catch block empty - at least log the problem
- Do not use exceptions for normal logic (use if conditions for simple checks)
- Prefer try-with-resources over manual finally for files and connections
- Throw the most meaningful exception: IllegalArgumentException instead of NullPointerException
- Use custom exceptions for business rules (SalException, InsufficientBalanceException)
- Do not catch Error (OutOfMemoryError, StackOverflowError)
- Show users friendly messages; keep stack traces in logs

Common Beginner Mistakes

- Parent catch before child catch: compile error (unreachable catch)
- Calling a checked-exception method without try-catch or throws
- Forgetting that code after the exception inside try is skipped
- Using e.printStackTrace() as the only handling in real applications
- Putting return inside finally (it hides exceptions)
- Multi-catch with related types, e.g. IOException | FileNotFoundException
- Forgetting to close files, scanners, connections
- Writing throw new Exception(...) when a specific type fits better

Practice Questions

- 1. What is the difference between an error and an exception?
- 2. Draw the hierarchy from Throwable down to NullPointerException and IOException.
- 3. Checked vs unchecked: give two examples of each.
- 4. What happens to the lines after the exception inside a try block?
- 5. Why must catch(Exception e) come after catch(ArithmeticException e)?
- 6. Difference between throw and throws? Write a small example.
- 7. When does finally NOT run? When does it run?
- 8. Rewrite FinallyExampleMain using try-with-resources.
- 9. Create InsufficientBalanceException and throw it from Account.withdraw().

Summary

- Exception = runtime problem that interrupts normal flow
- Hierarchy: Throwable -> Error / Exception -> RuntimeException (unchecked)
- Checked exceptions: handle with try-catch or declare with throws
- try-catch-finally: monitor, handle, cleanup; order catches child to parent
- throw raises, throws declares; custom exceptions extend Exception
- try-with-resources closes AutoCloseable resources automatically
- Unhandled exceptions propagate up the call stack and stop the program